Shortest path

Properties

Michel Bierlaire

Optimization: principles and algorithms

ÉCOLE POLYTECHNIQUE
FÉDÉRALE DE LAUSANNE

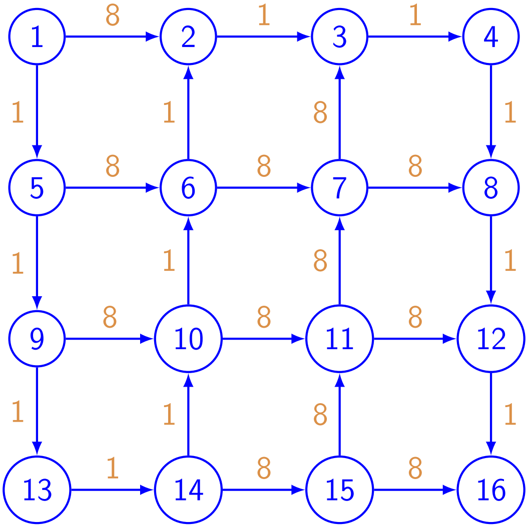
Properties

We have seen that the shortest path problem is a transhipment problem.
It could therefore be solved with the simplex algorithm.
However, it does not exploit the specific structure of the problem.
In this video, we identify some useful properties of the shortest path problem, that will be exploited by the algorithm.

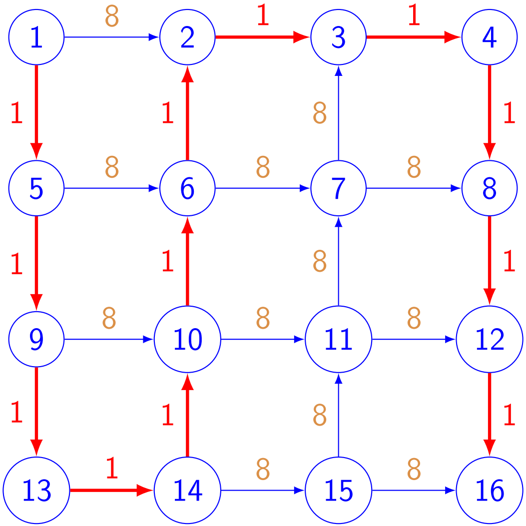
Intuition

When we look at a map, we have some intuition about what the shortest path could be.
It does not work in the general case
First, the computer has no bird eye's view on the network
Second, the triangle inequality is not always valid, especially when the coordinates of the nodes have no meaning at all.
In this example, there are 20 paths from node 1 to node 16.
We can identify the shortest by explicit enumeration.

Example



Example



Intuition

For a computer, the network is like a labyrinth

In order to figure out its way, it will write indications on the wall at each intersection

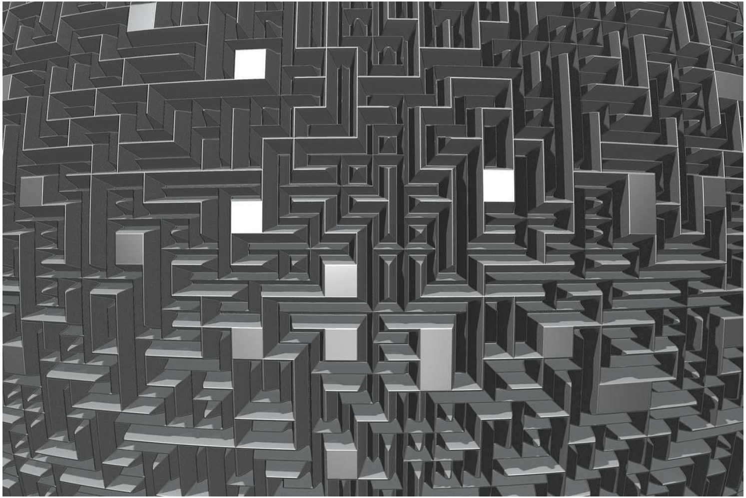
The idea is to record the distance done along various paths, and to write them down.

A value will be associated with each node.

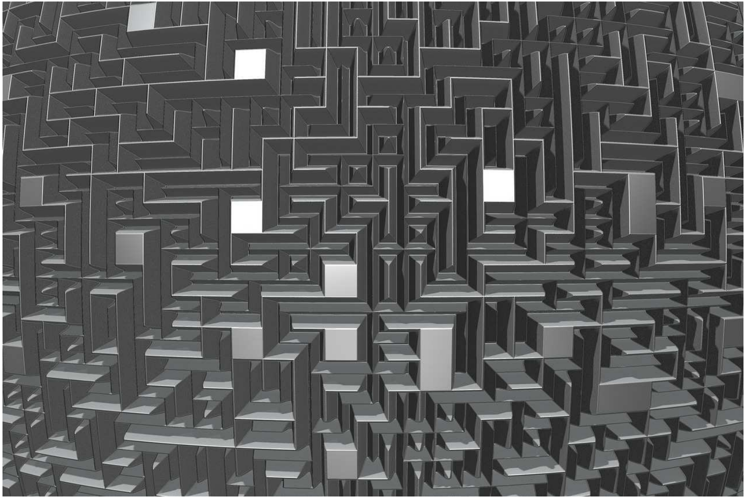
They will be called "labels"

Actually, these values happen to be the dual variables of the transhipment problem

Intuition



Intuition



Complementarity slackness for the transhipment problem

For all (i, j)

$$c_{ij} + \lambda_i - \lambda_j \geq 0.$$

For all (i, j) such that $x_{ij} > 0$

$$c_{ij} + \lambda_i - \lambda_j = 0.$$

Sufficient and necessary optimality conditions.

Optimality conditions

Theorem 23.1

Consider

- a network $(\mathcal{N}, \mathcal{A})$,$n$ arcs, m nodes,
- a cost vector $c \in \mathbb{R}^n$,
- a vector of labels $\lambda \in \mathbb{R}^m$,
- a path P between node o and node d.

$$\lambda_j \leq \lambda_i + c_{ij}, \quad \forall (i,j) \in \mathcal{A}.$$

$$\lambda_j = \lambda_i + c_{ij}, \quad \forall (i,j) \in P.$$

then P is a shortest path from o to d.

Proof

$$\lambda_j \leq \lambda_i + c_{ij}, \quad \forall (i,j) \in \mathcal{A}.$$

$$\lambda_j = \lambda_i + c_{ij}, \quad \forall (i,j) \in P.$$

Consider any path Q

$$Q = o \to j_1 \to j_2 \ldots j_\ell \to d.$$

$$C(Q) = c_{oj_1} + c_{j_1 j_2} + \ldots + c_{j_\ell d}.$$

$$C(Q) \geq (\lambda_{j_1} - \lambda_o) + (\lambda_{j_2} - \lambda_{j_1}) + \ldots + \lambda_d - \lambda_{j_\ell} = \lambda_d - \lambda_o,$$

$$P = o \to i_1 \to i_2 \ldots i_k \to d.$$

$$C(P) = c_{oi_1} + c_{i_1 i_2} + \ldots + c_{i_k d}.$$

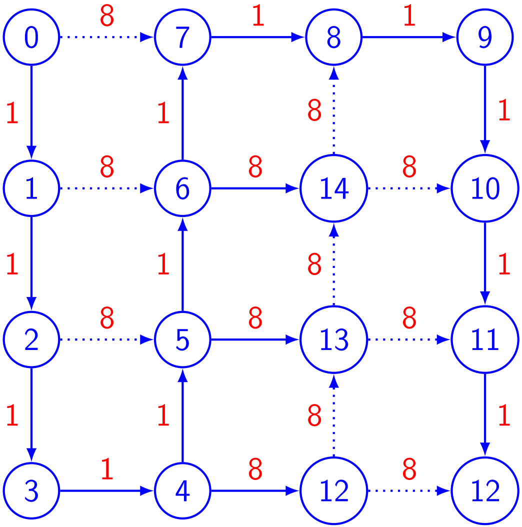
$$C(P) = (\lambda_{i_1} - \lambda_o) + (\lambda_{i_2} - \lambda_{i_1}) + \ldots + \lambda_d - \lambda_{i_k} = \lambda_d - \lambda_o.$$

Proof

Example

Show that the condition is verified with equality on the plain arcs
and with strict inequality on the dotted arcs
Show on an example that it corresponds to shortest paths
Mention that the subnetwork of plain arcs for a spanning tree.
Remind that, in a tree, there is exactly one path between two nodes.

Example: the shortest path tree



Labels

Optimality conditions

$$\lambda_j - \lambda_i \leq c_{ij}, \quad \forall (i,j) \in \mathcal{A}.$$

$$\lambda_j - \lambda_i = c_{ij}, \quad \forall (i,j) \in P.$$

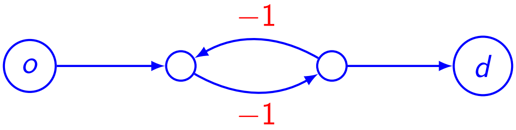
Only differences matter

Common practice: normalize $\lambda_o = 0$.

Negative cost cycle

If there exists a forward path from o to d containing a negative cost cycle, no forward path is the shortest path from o to d.

Negative cost cycle



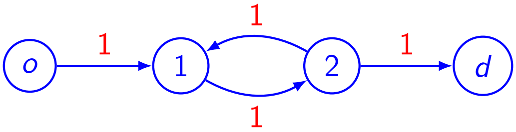
Simple paths

- *Consider a network* $(\mathcal{N}, \mathcal{A})$, *and*
- *two nodes* o *and* d.
- *Consider a forward path* P *between* o *and* d
- *that does not contain a negative cost cycle.*
- *Then there exists a simple forward path* Q *from* o *to* d *such that* $C(Q) \leq C(P)$.

Show path $o \to 1 \to 2 \to 1 \to 2 \to d$
Conclusion: if there is a shortest path, there is a shortest path that is simple.
Also lower bound on the length of shortest paths

$$C(P) \geq 0 \text{ if } c \geq 0. \text{ If not } C(P) \geq (m-1) \min_{(i,j)\in\mathcal{A}} c_{ij}.$$

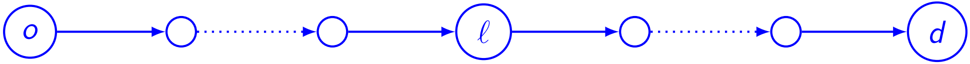
Simple paths



Principle of optimality

- *Consider a network* $(\mathcal{N}, \mathcal{A})$,
- *and two nodes* o *and* d.
- *Let* $P = o \rightarrow i_1 \rightarrow i_2 \ldots i_k \rightarrow d$ *be a shortest path from* o *to* d.
- *Then, for any* $\ell = 1, \ldots, k$, *the subpath* $P_{o\ell} = o \rightarrow \ldots \rightarrow i_\ell$ *is a shortest path from* o *to* i_ℓ
- *and* $P_{\ell d} = i_\ell \rightarrow \ldots \rightarrow d$ *is a shortest path from* i_ℓ *to* d.

Principle of optimality



Summary

Optimality conditions based on labels
Negative cost cycle generates an unbounded problem and no solution exists
If there is no negative cost cycle, we look only at simple paths
And there is always a shortest path.
Principle of optimality: partial paths are also optimal.